

# Understand the City, Improve Life

## A Privacy-First Smart City Data Platform

Turning camera footage → safe, useful city data

**AI Pipeline · Go Backend · Next.js Map**

# The Problem

Cities produce millions of images daily — but using this data faces two big obstacles:

- **Privacy risk** — footage contains faces, license plates, personal data (GDPR/KVKK)
- **Raw data is useless** — a video won't tell you "there's a damaged sign here"

**Question:** How do we understand the city through data *without* violating citizens' privacy?

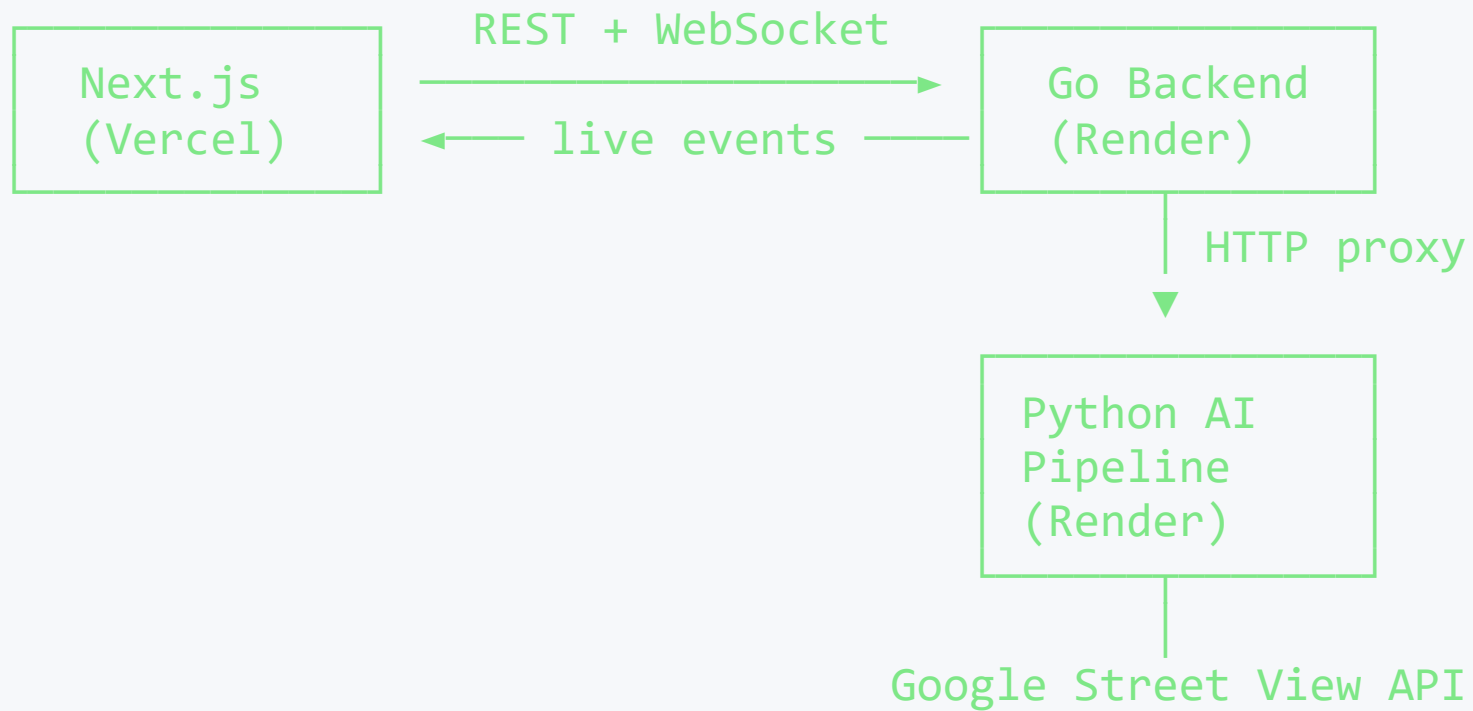
# Our Solution

A three-layer, end-to-end **privacy-first** data platform:

Footage → Anonymize → Detect → Serve → Show on Map  
(face/plate) (signs) (API)

1. **AI Pipeline** — anonymizes footage, detects urban objects
2. **Go Backend** — filters, enriches, serves data in real time
3. **Next.js Frontend** — displays it on an interactive map

# System Architecture



# 1. AI & Privacy Pipeline

**Fail-closed** design — if anonymization fails, the pipeline halts and never leaks raw data.

Street View footage

- face + license plate blur (irreversible)
- detection ONLY on anonymized frames
- remove duplicate detections
- JSON output + raw-data deletion proof

- Anonymization with **OpenCV** · detection with **YOLOv8**
- The model **never** sees the original footage

## GDPR / KVKK & Privacy (Critical)

"The model runs on **anonymized** video first, never on raw footage."

| Principle                      | Implementation                                          |
|--------------------------------|---------------------------------------------------------|
| <b>Purpose limitation</b>      | Urban object detection only — NO identity detection     |
| <b>Mandatory anonymization</b> | Faces + plates irreversibly blurred before training     |
| <b>Data minimization</b>       | JSON contains only: type, coordinates, confidence, time |
| <b>Deletion proof</b>          | Raw data auto-deleted and documented                    |

## JSON Output — The Only Data We Produce

```
{  
  "type": "damaged_sign",  
  "latitude": 41.021,  
  "longitude": 28.874,  
  "confidence": 0.91,  
  "timestamp": "00:01:24"  
}
```

No identity. No imagery. Just city data.

## 2. Go Backend — Production Grade

The **smart bridge** between the AI pipeline and the map:

- **Filterable API** — type, confidence score, geographic bounding box (bbox)
- **Stats endpoint** — type distribution, average confidence
- **WebSocket** — live scan tracking (started → running → completed)
- **Resilience** — rate limiting, panic recovery, graceful shutdown
- **30s cache** — serves data even if the pipeline briefly goes down

Only **3 external packages** — the rest is Go's standard library.



# API Endpoints

| Method | Path                     | Description               |
|--------|--------------------------|---------------------------|
| GET    | /health                  | Service + pipeline status |
| GET    | /api/v1/detections       | Filtered detection list   |
| GET    | /api/v1/detections/stats | Statistics                |
| POST   | /api/v1/scan             | Start a new scan          |
| GET    | /api/v1/scan/status      | Scan status               |
| GET    | /ws                      | WebSocket — live events   |

## Real-Time Experience

User picks a coordinate → scan starts → **watches live**:

```
{ "event": "scan_started",    "payload": {...} }  
{ "event": "scan_progress",  "payload": {"status": "running"} }  
{ "event": "scan_completed", "payload": {"detection_count": 5} }
```

Thanks to WebSocket, no "loading..." wait — **instant feedback**.

# Tech Stack

| Layer           | Technology                                          |
|-----------------|-----------------------------------------------------|
| Frontend        | Next.js · Vercel                                    |
| Backend         | Go (chi, gorilla/websocket) · Render                |
| AI Pipeline     | Python · OpenCV · YOLOv8 · Flask · Render           |
| External API    | Google Street View API                              |
| Deployment      | Docker (multi-stage, ~7 MB image)                   |
| Dev Environment | Cursor IDE · agentic ruleset · Git feature branches |

## Deployment — Live

Both services run live on **Render.com** via Docker:

- **Go Backend** → `cursor-j7jx.onrender.com`
  - `scratch` image · ~7 MB · near-zero attack surface
- **Python Pipeline** → `ai-privacy-pipeline.onrender.com`
  - Gunicorn · health checks · automatic sample fallback

The health endpoint reports both services' status live.

# Live Demo

# 1. Is the service up?

```
curl .../health
```

```
→ { "status": "ok", "pipeline_status": "ok" }
```

# 2. Filter damaged signs with high confidence

```
curl ".../api/v1/detections?type=damaged_sign&min_confidence=0.8"
```

# 3. Dashboard statistics

```
curl .../api/v1/detections/stats
```

```
→ { "total": 5, "by_type": {...}, "avg_confidence": 0.8 }
```

Then: live scan over WebSocket → pins appearing on the map.

## Why This Project Matters

- **Social benefit** — safer cities via damaged-sign and pothole detection
- **Privacy-first** — GDPR/KVKK compliance built into the architecture, not bolted on
- **Data-driven decisions** — real coordinate-based data, not guesswork
- **Scalable** — can grow into a real smart-city service

Real value begins when code leaves the screen and touches a human life.

# Thank You

We welcome your questions

Understand the City · Improve Life · Protect Privacy